File Handling

Eunji Kim
eunjikim@cau.ac.kr

File Handling

File handling is an important part of web application and data analytics.

Python has several functions for creating, reading, updating, and deleting files.

Text File: `demofile.txt`

```
Hello! Welcome to demofile.txt
This file is for testing purposes.
Good Luck!
```

1. Open, Read, and Close a File

To work with a file, you first need to open it using the built-in `open()` function. This function returns a file object, which has methods like `read()` to access its content.

Example:

```
f = open("./data/demofile.txt")
print(f.read())
```

Output:

```
Hello! Welcome to demofile.txt
This file is for testing purposes.
Good Luck!
```

> 📝 **Note:** Make sure the file exists, or you will get an error.

Reading a Portion of a File

By default, the `read()` method returns the whole text. You can also specify how many characters to return.

Example: Read the first 10 characters

```
with open("./data/demofile.txt") as f:
    # Returns the 10 first characters of the file
    print(f.read(10))
```

Output:

```
Hello! Wel
```

Specifying File Paths

If the file is not in the same directory, you must specify its path.

Relative Path (Parent Folder):

```
# The ../ moves up one directory level
f = open("../welcome.txt")
print(f.read())
```

Absolute (Full) Path:

```
# The full path from the root directory

# windows
f = open("C:\\Users\\dmlab\\Downloads\\welcome.txt")
print(f.read())


# mac
f = open("/Users/dmlab/Downloads/welcome.txt")
print(f.read())
```

Closing Files

It is good practice to **always close the file** when you are done with it using the `close()` method.

```
f = open("./data/demofile.txt")
print(f.read())
f.close() # Closes the file
```

> 📝 **Note:** In some cases, due to buffering, changes made to a file may not show until you close the file.

2. Using the `with` statement

The `with` statement is the recommended way to open files. It automatically handles closing the file for you, even if errors occur.

Example:

```
with open("./data/demofile.txt") as f:
    print(f.read())
```

Output:

```
Hello! Welcome to demofile.txt
This file is for testing purposes.
Good Luck!
```

File Open

The key function for working with files in Python is the `open()` function.

The `open()` function takes two parameters; ***filename***, and ***mode***.

```
with open('./data/demofile.txt', 'r') as f:
    # ...
```

There are four different methods (modes) for opening a file:

- `"r"` - Read - **Default** value. Opens a file for reading, error if the file does not exist
- `"a"` - Append - Opens a file for appending, creates the file if it does not exist
- `"w"` - Write - Opens a file for writing, creates the file if it does not exist
- `"x"` - Create - Creates the specified file, returns an error if the file exists

3. Write to an Existing File

To write to a file, you must add a second parameter to the `open()` function, specifying the **mode**.

- `"a"` - **Append**: Adds content to the end of the file.
- `"w"` - **Write**: Overwrites any existing content in the file.

Appending to a File (`"a"`)

Using `"a"` mode adds new content without deleting what's already there.

1. Append new content:

```
with open("./data/demofile.txt", "a") as f:
  f.write("Now the file has more content!")
```

2. Read the file to see the result:

```
with open("./data/demofile.txt", "r") as f: # "r" is for read
  print(f.read())
```

Output:

```
Hello! Welcome to demofile.txt
This file is for testing purposes.
Good Luck!Now the file has more content!
```

Overwriting a File (`"w"`)

Using `"w"` mode will erase the file's current content and write new content.

1. Overwrite the content:

```
with open("./data/demofile.txt", "w") as f:
  f.write("Woops! I have deleted the content!")
```

2. Read the file to see the result:

```
with open("./data/demofile.txt") as f: # "r" is the default mode
  print(f.read())
```

Output:

```
Woops! I have deleted the content!
```

4. Create a New File

You can create a new file using `open()` with one of these modes:

- `"x"` - **Create**: Creates a file. Returns an error if the file already exists.
- `"a"` - **Append**: Creates the file if it does not exist.
- `"w"` - **Write**: Creates the file if it does not exist.

Creating a File (`"x"`)

The `"x"` mode is used exclusively to create a new file.

Example:

```
# This creates a new, empty file named "myfile.txt"
f = open("myfile.txt", "x")
f.close()
```

> 📝 **Note:** If `myfile.txt` already exists, this code will raise a `FileExistsError`.

Thank You.